

Vigil - Technology Stack & Product Guide

Smart India Hackathon 2026 - Problem Statement 26189

AI-Powered Criminal Network Analysis System

GitHub: github.com/SiddharthDC786/investigation-ai

Document purpose: Share with judges, mentors, or collaborators - explains every technology, its role in Vigil, and why it was chosen.

1. What Is Vigil?

Vigil is a court-defensible investigation console for police and cyber-crime units. It fuses FIR documents, call detail records (CDR), bank transactions, surveillance notes, and lawful OSINT into one searchable system.

Problems Vigil Solves

Problem	Real-world impact
Same name, many people	"Rahul" may match 50 records - which one is the suspect?
FIR spelling errors	"Mukkherjee" vs "Mukherjee" breaks manual search
Data in silos	CDR in Excel, bank in another file, FIR as PDF
Hidden connections	Suspects never call directly but share a prepaid SIM
No audit trail	OSINT lookups cannot be proven in court
Black-box AI	Investigators cannot explain why someone was flagged

Three Product Pillars

- 1. **Data fusion** - One person-centric view across all sources (inspired by Palantir Gotham)
- 2. **Network analytics** - Graph, timeline, priority scoring (inspired by IBM i2 Analyst's Notebook)
- 3. **Lawful OSINT + audit** - Logged enrichments with tamper-evident hash chain (inspired by Social Links)

2. System Architecture



Important: When the frontend .env sets VITE_API_BASE_URL=http://localhost:8000, search, timeline, risk scores,

ingest, and OSINT use real PostgreSQL queries - not mock data.

3. Frontend Technologies

Technology	Role in Vigil	Why we use it
React 19	Builds all UI views: Person Search, Connection Map, Event Timeline, Priority List, Record Search, Action Log	Industry standard; component reuse; fast updates for complex dashboards
TypeScript	Types for API responses (search results, risk scores, timeline events)	Catches bugs before runtime; shared contracts with backend
Vite 8	Dev server and production bundle	Very fast hot reload during hackathon development
Tailwind CSS v4	Dark "investigation console" styling	Consistent police-grade UI without writing thousands of CSS lines
react-force-graph-2d	Connection Map - persons, phones, call-weighted edges	Interactive network layout; suspect-centered demo story
Recharts	Priority List horizontal bar chart	Shows evidence-based scores clearly for
Framer Motion	Subtle UI transitions	Professional feel without heavy animation libraries
IBM Plex fonts	Typography across the app	Readable monospace + sans for data-heavy screens
i18n (English + Hindi)	Bilingual labels and hints	SIH requirement; usable across Indian states

Where Each Frontend Piece Appears

File / area	Part in project
`SearchView.tsx`	Person search, disambiguation, live filters, matched-person card
`NetworkGraphView.tsx`	CDR-based connection map with evidence on links
`TimelineView.tsx`	Chronological events filtered by selected person
`RiskScoringView.tsx`	Evidence-based priority chart and ranked list
`OsintView.tsx`	Person dossier + lawful record lookups
`AuditTrailView.tsx`	Action log with category badges and hash-chain status
`IngestPanel.tsx`	FIR text/OCR ingest with review-before-upload
`InspectorPanel.tsx`	Selected entity detail + AI explainability
`api/*.ts`	HTTP clients calling FastAPI endpoints

Why web, not desktop app? Investigators already use browsers. No extra install - only backend + database on a district PC.

4. Backend Technologies

Technology	Role in Vigil	Why we use it
Python 3.12	All server logic, NLP, graph analytics	Best ecosystem for data science + NLP + police analytics
FastAPI	REST API: <code>`/search`</code> , <code>`/timeline`</code> , <code>`/ingest`</code> , <code>`/analyze`</code> , <code>`/osint`</code>	Auto OpenAPI docs, async-ready, fast to build routers
Uvicorn	ASGI server hosting FastAPI	Production-grade, standard for FastAPI
SQLAlchemy 2	Database session and ORM patterns	Reliable PostgreSQL access alongside raw
psycopg 3	PostgreSQL driver	Modern async-capable driver for crime_network DB
Pydantic	Request/response validation	Strict API schemas; fewer integration bugs

Backend Service Map

Service file	Part in project
<code>`investigation_search.py`</code>	Person search, fuzzy names, filter intersection, related people
<code>`case_graph.py`</code>	Investigation connection map from CDR
<code>`timeline_service.py`</code>	Merges CDR, transactions, FIR, chat into timeline
<code>`risk_score_service.py`</code>	PageRank + role + CDR/FIR evidence scoring
<code>`ingest_service.py`</code>	FIR ingest, entity resolution, DB merge
<code>`ocr_service.py`</code>	FIR photo to cleaned text (Tesseract)
<code>`osint_enrichment.py`</code>	Lawful lookup simulation + audit chain
<code>`explain_service.py`</code>	Citations and reasoning for selected entity
<code>`audit_chain.py`</code>	SHA-256 tamper-evident log for OSINT

Why FastAPI over Django? Lighter weight; each feature is a focused router; ideal for hackathon iteration.

5. Database Technologies

Technology	Role in Vigil	Why we use it
PostgreSQL 16	Primary store: people, phones, CDR, transactions, FIR, relationships	ACID compliance; SQL joins for search/timeline; court-grade persistence
Neo4j 5.26 (optional)	Graph fusion, cross-source full-text fallback	Demonstrates graph-native linking; app works without it

Key PostgreSQL Tables

Table	Part in project
`people`	Canonical identity: name, DOB, gender, city
`recorded_names`	FIR typos, aliases, maiden names for fuzzy search
`phones`	Links persons to phone numbers for CDR join
`cdr`	Call records - powers connection map and timeline
`bank_accounts` + `transactions`	Money flow events on timeline
`fir`	Complaint text - NLP ingest and FIR mention scoring
`relationships`	Handler, associate, witness roles per case
`audit_chain`	Tamper-evident OSINT log entries

Why PostgreSQL first? Police data is naturally rows and columns. One SQL query can join a person to their calls, accounts, and FIR mentions.

Why optional Neo4j? Shows advanced graph fusion to judges; core demo path uses PostgreSQL CDR graph.

6. AI, NLP & Analytics (Explainable - Not Black-Box LLM)

Technology	Role in Vigil	Why we use it
spaCy (`en_core_web_sm`)	Extract PERSON, ORG, GPE from FIR text on ingest	Deterministic NER; results citeable in court
Regex patterns	Indian phone (+91), bank account, vehicle numbers	High precision on structured identifiers
RapidFuzz	Fuzzy name match (typos, partial names) in search + ingest	Solves "Mukherjee" vs "Mukherjee" without opaque ML
NetworkX	PageRank, betweenness, Louvain	Standard graph algorithms for priority and
SciPy	Sparse matrix support for centrality	Efficient computation on large call graphs
Pillow + Tesseract	OCR: FIR photo to editable text	Officers upload scans; human verifies before DB save

Why Not ChatGPT for Everything?

Court testimony requires citations - "matched recorded_names row X at 92% confidence." Black-box LLM answers are hard to defend. Vigil uses transparent pipelines with /explain endpoints and structured audit logs.

7. Data Pipeline & Infrastructure

Component	Role in Vigil	Why we use it
<code>**generate_dataset.py**</code>	Creates 240 synthetic cases, 60K+ CDR, 25K transactions	Safe demo without real PII; Faker library
<code>**import_to_db.py**</code>	Loads CSVs into PostgreSQL	One-command database seed
<code>**seed_demo_rahuls.sql**</code>	Multiple "Rahul" suspects/witnesses in CASE0001	Demo name disambiguation for judges
<code>**Docker Compose**</code>	Postgres + Neo4j + backend containers	One-command setup for teammates
<code>**GitHub**</code>	github.com/SiddharthDC786/investigation-ai	Version control and team collaboration

8. Who Are the Users?

User	Role	How they use Vigil
<code>**Field investigator (IO)**</code>	First-line case work	Search, disambiguate names, view connection map
<code>**Cyber crime analyst**</code>	CDR / digital forensics	Call graph, shared-contact bridges, timeline
<code>**OSINT officer**</code>	Lawful public lookups	Record Search dossier + logged enrichments
<code>**Case supervisor**</code>	Oversight and disclosure	Action log, hash verification, JSON export
<code>**Judges / evaluators**</code>	SIH presentation	End-to-end demo: search to audit

Primary persona: A district cyber cell officer with FIR PDFs, Excel CDR, and bank CSVs who needs one screen instead of five tools.

9. How Vigil Helps Investigators

- `**Same-name resolution**` - Search name + city + phone; pick suspect vs witness vs handler.
- `**Multi-source fusion**` - One timeline filtered by selected person (calls, money, FIR).
- `**Hidden links**` - Connection map shows CDR-weighted edges and shared-contact bridges.
- `**Evidence-based priority**` - Priority List ranks by centrality + role + CDR/FIR evidence.
- `**FIR ingest with human check**` - OCR to editable text, preview entities, then upload to database.
- `**Court-defensible audit**` - OSINT hash chain; searches and ingests logged with officer ID.
- `**Explainability**` - Every flag cites `people.csv`, `recorded_names`, CDR count, or FIR text.
- `**Offline capable**` - Core stack runs locally without cloud APIs.

10. How Vigil Is Better Than Alternatives

Compared to	Vigil advantage
Excel + manual search	Fuzzy names, graph links, auto timeline, role inference
Palantir Gotham / IBM i2	Same workflows at zero license cost ; open source stack
Social Links / OSINT SaaS	Built-in tamper-evident audit chain
Generic police RMS	Built for network crimes - CDR graphs, alias tables
ChatGPT on FIR PDFs	Deterministic NER + RapidFuzz + cited <code>`explain`</code>
Single-purpose CDR tools	Unified person-centric search, map, timeline, dossier, log

Key Differentiators for SIH

- ? Identity resolution for Indian FIR typos and duplicate names
- ? Officer-in-the-loop disambiguation (algorithm suggests, human confirms)
- ? Open stack - no vendor lock-in
- ? Bilingual UI
- ? Synthetic realistic dataset for safe demos

11. Feature-to-Technology Map

App feature	Technologies involved
Person Search	React, FastAPI, PostgreSQL, RapidFuzz, recorded_names
Connection Map	react-force-graph-2d, case_graph.py, CDR in PostgreSQL
Event Timeline	TimelineView, timeline_service.py, person-filtered events
Priority List	Recharts, NetworkX, risk_score_service.py
Record Search (OSINT)	OsintView, explain_service.py, audit_chain.py
FIR Upload	Tesseract OCR, spaCy NER, ingest_service.py
Action Log	AuditTrailView, session log + SHA-256 chain
Identity explainability	explain_service.py, citations from DB

12. Ports, Credentials & Demo Data

Item	Value
Frontend URL	http://localhost:5173
Backend URL	http://localhost:8000
PostgreSQL	localhost:5432 / database: crime_network
Neo4j Browser (optional)	http://localhost:7474
Login	INV-2847 / vigil2026
Demo case	CASE0001 (display: FIR-042/2026)
Main suspect	P00014 - Rahul Mukherjee, Hyderabad
GitHub	github.com/SiddharthDC786/investigation-ai

13. Honest Limitations

Area	Current status	Production fix
Demo auth	Client-side login	JWT / government SSO
Face search	Fixed demo match	Real biometrics API
OSINT sources	Synthetic enrichments	Licensed government APIs
Neo4j	Optional, often off in demo	Bundled in Docker Compose
English/Hindi only	i18n ready	Add regional languages

14. One-Paragraph Elevator Pitch

Vigil is an open-source criminal investigation console for Smart India Hackathon PS 26189. Built with React, FastAPI, PostgreSQL, spaCy, RapidFuzz, and NetworkX, it fuses FIR documents, call records, and bank data into one searchable system - solving same-name confusion, FIR typos, and hidden phone bridges. Unlike crore-priced commercial tools, Vigil runs on a district laptop, explains every flag with citations, and logs every OSINT action in a tamper-evident audit chain for court disclosure.

Vigil Team - Smart India Hackathon 2026 - Confidential team documentation